

BUILDING REST APIs WITH DJANGO

PARTICIPANT ENVIRONMENT GUIDE

PROGRAM SUPPORT

TOOLS.
INSTALL.
VERIFY.

ORGANIZATION	PROGRAM	SUPPORTED SYSTEMS	STATUS
Tuwaiq Club at KFUPM	Building REST APIs with Django	Windows + macOS	Ready for external validation

PURPOSE

ONE CONSISTENT, ZERO-COST WORKSHOP ENVIRONMENT

This guide tells participants exactly what to install, how to verify every tool, how to prepare the Django project environment, and how the database changes from local SQLite to hosted PostgreSQL. All required tools and service plans listed here can be used at no cost for this program.

APPROVED STACK

WORK	SELECTED TOOL	USE IN THE PROGRAM
Code	Python 3.13 + VS Code	Write, run, debug, and test Django code
Web API	Django 5.2 LTS + DRF 3.16	Build models, serializers, endpoints, auth, and tests
Local data	SQLite + DB Browser	Simple local database and visual inspection
API requests	Postman Desktop	Send requests, tokens, JSON bodies, and collections
Source	Git + GitHub Free	Version history, submission, and deployment source
Production	Render Free + Neon PostgreSQL Free	Deploy the API and use a persistent production database

IMPORTANT

Complete the installation and final readiness checklist **before Day 1**. Cloud free tiers can change; the instructor should re-check Render and Neon availability one week before delivery.

ENTRY PREREQUISITES AND WHAT TO BRING

The program is beginner-friendly, but participants need a small foundation before the first session.

MINIMUM REQUIRED KNOWLEDGE

AREA	MINIMUM LEVEL	A PARTICIPANT SHOULD BE ABLE TO...
Python	Beginner	Use variables, strings, numbers, lists, dictionaries, conditions, loops, and simple functions
Files and folders	Beginner	Create a folder, locate a downloaded file, and recognize a file path
Terminal	Beginner	Open PowerShell or Terminal, run a command, and change into a project folder
Web basics	Awareness	Recognize a URL, request, response, HTTP method, and status code
JSON	Awareness	Read a small JSON object containing names, numbers, booleans, and lists
Git	Helpful, not required	Prior commits are useful, but Git commands are taught and practiced during the program

NOT REQUIRED BEFORE JOINING

Participants do **not** need previous Django, Django REST Framework, database administration, cloud deployment, or professional backend experience. These are taught in the program.

BRING ON DAY 1

- The same prepared laptop used for the readiness check
- Laptop charger and, if needed, a plug adapter
- Phone or authentication device for two-factor codes
- Access to the account email and verification codes
- Readiness evidence and any unresolved error text

COMPLETE BEFORE DAY 1

- Install every required tool in this guide
- Create and verify the required free accounts
- Complete the knowledge and environment checks
- Submit the evidence listed in the readiness section
- Attend the setup clinic if any required check fails

PARTICIPATION RULE

A participant may join with beginner-level knowledge, but must arrive with a working environment or an approved recovery environment. Day 1 is for building the API, not for installing software.

WHAT EVERY PARTICIPANT NEEDS

Required downloads, accounts, and a recommended installation sequence.

REQUIRED DOWNLOADS

TOOL	REQUIRED VERSION / EDITION	COST	WHY
Python	Latest Python 3.13.x, 64-bit	Free	Program runtime and package manager
VS Code	Current Stable release	Free	Shared editor and terminal experience
VS Code extensions	Python, Pylance, Python Debugger	Free	Interpreter selection, completion, debugging, tests
Git	Current stable release	Free	Commits, branches, and GitHub
Postman	Desktop app, Free plan	Free	Manual API testing and collections
DB Browser	DB Browser for SQLite stable	Free	Inspect the local SQLite file

FREE ACCOUNTS

- **GitHub:** repository hosting and submission.
- **Postman:** optional for basic requests, recommended for collections.
- **Neon:** hosted PostgreSQL database for the deployed API.
- **Render:** hosted Django web service connected to GitHub.

LAPTOP REQUIREMENTS

- Windows 10/11 64-bit or a supported macOS release
- Administrator access for application installation
- At least 8 GB RAM and 5 GB available disk space recommended
- Reliable internet access and a modern browser
- Access to the email used for verification codes

INSTALL IN THIS ORDER

- 1 Install Python, then verify the command works.
- 2 Install VS Code and its three Python extensions.
- 3 Install Git and configure your identity.
- 4 Install Postman Desktop and DB Browser for SQLite.
- 5 Create or verify GitHub, Neon, and Render accounts.
- 6 Run the complete readiness check on the final pages.

NOT REQUIRED

Do not install Docker, Node.js, a paid IDE, a local PostgreSQL server, or an AI coding subscription. They add setup risk without helping the required five-day project.

WINDOWS SETUP - PYTHON AND VS CODE

Use a normal Windows user account with permission to install applications.

1. INSTALL PYTHON 3.13

- 1 Open the official Python downloads page and choose the latest **Python 3.13.x Windows installer (64-bit)**.
- 2 Run the installer. If shown, enable **Add python.exe to PATH**. Keep **pip** and **venv** selected.
- 3 Finish the installation, close existing terminals, and open a new PowerShell window.
- 4 Run the verification commands below. The Python output must begin with 3.13 and pip must show a path on your computer.

[Official Python downloads for Windows](#)

```
py --version
py -m pip --version
```

IF 'PY' IS NOT RECOGNIZED

Restart Windows first. If the command still fails, rerun the Python installer and enable the PATH/launcher options. Do not download Python from an unofficial website.

2. INSTALL VISUAL STUDIO CODE

- 1 Download the **Windows User Installer** from the official VS Code website.
- 2 Run the installer. Enable **Add to PATH**, **Open with Code**, and **Register Code as an editor** when available.
- 3 Open VS Code and select the Extensions icon on the left.
- 4 Install the extensions named **Python**, **Pylance**, and **Python Debugger**, all published by Microsoft.
- 5 Open **Terminal > New Terminal** and repeat `py --version`.

[Official VS Code download](#)

EXTENSION	PUBLISHER	PURPOSE
Python	Microsoft	Python support, testing, environments, and tooling
Pylance	Microsoft	Autocomplete, navigation, and type/error feedback
Python Debugger	Microsoft	Breakpoints and step-by-step debugging

PASS CONDITION

VS Code opens successfully, the three extensions show **Installed**, and the integrated terminal recognizes Python 3.13.

WINDOWS SETUP - GIT, POSTMAN, AND SQLITE

Complete all three installations before creating the workshop project.

3. INSTALL AND CONFIGURE GIT

- 1 Download the latest **64-bit Git for Windows** from git-scm.com.
- 2 Run the installer. Keep the default options unless the instructor provides a program-specific setting.
- 3 Allow Git to use the bundled credential manager. This simplifies GitHub authentication.
- 4 Open a new PowerShell or VS Code terminal and verify Git.
- 5 Configure the name and email that should appear in GitHub commits.

[Official Git for Windows](https://git-scm.com/download/win)

```
git --version
git config --global user.name "Your Full Name"
git config --global user.email "your-email@example.com"
git config --global init.defaultBranch main
git config --global --list
```

4. INSTALL POSTMAN DESKTOP

- 1 Download the Windows x64 Postman Desktop app from the official Postman website.
- 2 Install and open Postman. Basic API requests work through the lightweight client without signing in.
- 3 For saved/synchronized collections, create or sign in to a free individual account.
- 4 Send the verification request below. A successful response confirms networking and the app are working.

[Official Postman download](https://www.postman.com/downloads/)

```
Method: GET
URL: https://postman-echo.com/get
Expected status: 200 OK
```

5. INSTALL DB BROWSER FOR SQLITE

- 1 Download the latest standard **Windows 64-bit installer** from sqlitebrowser.org.
- 2 Complete the installation using the default destination.
- 3 Open DB Browser and confirm the menus **Open Database**, **Database Structure**, and **Browse Data** are visible.
- 4 Do not create a database manually. Django creates `db.sqlite3` on Day 1.

[DB Browser for SQLite downloads](https://sqlitebrowser.org/downloads/)

WINDOWS PASS CONDITION

- `git --version` succeeds.
- Postman returns 200 from Postman Echo.
- DB Browser opens without an installation warning.

MACOS SETUP - PYTHON AND VS CODE

Do not use the system Python supplied by macOS for this project.

1. INSTALL PYTHON 3.13

- 1 Open the official Python downloads page and download the latest **Python 3.13.x macOS 64-bit universal2 installer**.
- 2 Open the downloaded .pkg and complete the standard installation.
- 3 Open Terminal from Applications > Utilities.
- 4 Run the verification commands below. Use python3 before the virtual environment is activated.

[Official Python downloads for macOS](#)

```
python3 --version  
python3 -m pip --version
```

APPLE SILICON AND INTEL

The universal2 Python installer supports current Apple silicon and Intel Macs. When downloading other applications, choose **Apple Silicon** for M-series Macs and **Intel** for older Intel Macs.

2. INSTALL VISUAL STUDIO CODE

- 1 Download the correct macOS build from the official VS Code page.
- 2 Open the archive and drag **Visual Studio Code.app** into Applications.
- 3 Open VS Code. If macOS displays a security prompt, confirm the application was downloaded from Microsoft.
- 4 Install the Microsoft **Python**, **Pylance**, and **Python Debugger** extensions.
- 5 Press Command+Shift+P, run **Shell Command: Install 'code' command in PATH**, then restart Terminal.

[Official VS Code macOS instructions](#)

```
code --version  
python3 --version
```

PASS CONDITION

VS Code opens from Applications and from the code terminal command, all three extensions are installed, and Terminal reports Python 3.13.

MACOS SETUP - GIT, POSTMAN, AND SQLITE

Use the installer method below; Homebrew remains optional.

3. INSTALL AND CONFIGURE GIT

- 1 In Terminal, run `git --version`. macOS may offer to install Command Line Developer Tools; accept the installation if prompted.
- 2 If no prompt appears and Git is unavailable, use the installer options listed on git-scm.com.
- 3 Configure your commit name, GitHub email, and default branch.
- 4 Verify the saved configuration.

[Official Git installation options for macOS](#)

```
git --version
git config --global user.name "Your Full Name"
git config --global user.email "your-email@example.com"
git config --global init.defaultBranch main
git config --global --list
```

4. INSTALL POSTMAN DESKTOP

- 1 Download the Postman build that matches your Mac: Apple Silicon or Intel.
- 2 Move Postman into Applications and open it.
- 3 Use the lightweight client without signing in, or use a free account to synchronize collections.
- 4 Send the verification request below.

[Official Postman download](#)

```
Method: GET
URL: https://postman-echo.com/get
Expected status: 200 OK
```

5. INSTALL DB BROWSER FOR SQLITE

- 1 Download the latest macOS package from sqlitebrowser.org.
- 2 Move DB Browser for SQLite into Applications when prompted.
- 3 Open the application. If Gatekeeper blocks it, confirm the download came from sqlitebrowser.org and allow it through Privacy & Security.
- 4 Do not create a database manually; open Django's `db.sqlite3` after Day 1 migrations.

[DB Browser for SQLite downloads](#)

MACOS PASS CONDITION

- `git --version` succeeds.
- Postman returns 200 from Postman Echo.
- DB Browser opens from Applications.

CREATE THE FREE SERVICE ACCOUNTS

Use one professional email address where possible and enable multi-factor authentication.

SERVICE	PURPOSE	REQUIRED BEFORE	FREE-LIMIT NOTE
GitHub	Repository, commits, submission, Render connection	Day 1	GitHub Free includes unlimited public/private repositories
Postman	Sync the Task API collection	Day 2	Free individual plan is sufficient
Neon	Hosted PostgreSQL production database	Day 4	Use one small project on the \$0 plan
Render	Host the deployed Django API	Day 4	Free web services sleep after inactivity

GITHUB

- 1 Create or sign in to a personal GitHub account at github.com.
- 2 Verify the email address and enable two-factor authentication.
- 3 Create a temporary private repository named `set-up-check` to confirm repository creation works; it may be deleted later.
- 4 In VS Code, use the Accounts icon to sign in to GitHub when prompted.

[GitHub account signup](#)

NEON AND RENDER

NEON

- 1 Create a free account.
- 2 Confirm the dashboard opens.
- 3 Do not create/share production credentials yet; this happens during deployment.

RENDER

- 1 Create a free account.
- 2 Connect the same GitHub account when authorized.
- 3 Confirm the New Web Service screen can see eligible repositories.

[Neon](#)

[Render](#)

NEVER COMMIT CREDENTIALS

Database URLs, Django secret keys, tokens, and passwords must never appear in Git commits, screenshots, Postman public workspaces, or shared chat messages.

CREATE AND VERIFY THE READINESS ENVIRONMENT

This temporary folder validates Python, virtual environments, packages, Django, Git, and VS Code before Day 1.

WINDOWS POWERSHELL

```
mkdir django-readiness-check
cd django-readiness-check
py -m venv .env
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install "Django>=5.2,<5.3" "djangorestframework>=3.16,<3.17"
python -m django --version
code .
```

MACOS TERMINAL

```
mkdir django-readiness-check
cd django-readiness-check
python3 -m venv .env
source .env/bin/activate
python -m pip install --upgrade pip
python -m pip install "Django>=5.2,<5.3" "djangorestframework>=3.16,<3.17"
python -m django --version
code .
```

SELECT THE CORRECT PYTHON INTERPRETER

- 1 In VS Code, press Ctrl+Shift+P on Windows or Command+Shift+P on macOS.
- 2 Run **Python: Select Interpreter**.
- 3 Choose the interpreter inside the project folder named `.env`.
- 4 Open a new VS Code terminal. It should display `(.env)` at the beginning of the prompt.

CREATE A TEST DJANGO PROJECT

```
django-admin startproject config .
python manage.py migrate
python manage.py runserver
```

EXPECTED RESULT

Open `http://127.0.0.1:8000/`. The Django success page should load. Stop the server with Ctrl+C. If this works, the core development environment is ready. Keep this folder until the setup evidence is approved; it is not the five-day project.

WINDOWS ACTIVATION ERROR

If PowerShell blocks `Activate.ps1`, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, confirm the change, close PowerShell, reopen it, and activate the environment again.

PRE-PROGRAM READINESS EXERCISE

Complete this independently in approximately 20 minutes after all installations are finished.

PART A - QUICK KNOWLEDGE CHECK

QUESTION	YOUR ANSWER
Which HTTP method normally reads a collection without changing it?	_____
Which HTTP method normally creates a new resource?	_____
What does a 201 HTTP status normally mean?	_____
In JSON, is false a string, number, or boolean?	_____

ANSWER KEY

GET; POST; the resource was created successfully; boolean. A participant who misses an answer may still join, but should review HTTP methods and JSON before Day 1.

PART B - PRACTICAL CHECK

- 1 Open the `django-readiness-check` folder and activate its `.venv` environment.
- 2 Create a file named `readiness_check.py` with the code below, then run it.
- 3 Run `python manage.py check`; the result must report no identified issues.
- 4 Start Django and open `http://127.0.0.1:8000/` in a browser.
- 5 Send the Postman Echo request and confirm that Postman returns HTTP 200.

```
# readiness_check.py
import json

task = {"title": "Environment check", "completed": False}
print(json.dumps(task))

# Run:
python readiness_check.py
```

COMPLETION CRITERIA

The script prints valid JSON, Django reports no issues, the local success page loads, and Postman returns 200. Submit the screenshots or copied terminal output requested in the final readiness section.

HOW THE PARTICIPANT PROJECT IS CREATED

The approved program path is to create the Task Management API from scratch on Day 1; no starter repository is required.

BEFORE DAY 1 VS. DAY 1

Before Day 1, use only the django-readiness-check folder. On Day 1, create a separate task-management-api folder and repository with the instructor. This preserves the learning objective of building the project structure together.

DAY 1 - CREATE THE LOCAL PROJECT

```
# Windows PowerShell
mkdir task-management-api
cd task-management-api
py -m venv .venv
.venv\Scripts\Activate.ps1

# macOS Terminal uses these two lines instead:
# python3 -m venv .venv
# source .venv/bin/activate

python -m pip install "Django>=5.2,<5.3" "djangorestframework>=3.16,<3.17"
django-admin startproject config .
python manage.py startapp tasks
python manage.py migrate
python manage.py runserver
```

DAY 1 - CREATE AND CONNECT THE GITHUB REPOSITORY

- 1 Create an empty GitHub repository named task-management-api. Do not add a README, license, or .gitignore on GitHub.
- 2 Create the local .gitignore entries shown on the next page before staging files.
- 3 Initialize Git, commit the generated project, and connect the remote URL belonging to your account.
- 4 Push the main branch and confirm the files appear on GitHub.

```
git init
git add .
git commit -m "Initialize Django project"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/task-management-api.git
git push -u origin main
```

IF A STARTER REPOSITORY IS APPROVED LATER

The Curriculum Lead must replace this create-from-scratch path with one official repository URL and test the clone instructions before publishing the guide. Participants must never guess a repository link or download workshop files from an unofficial source.

INSTALL THE COMPLETE PROJECT PACKAGE SET

Install packages inside the activated virtual environment, never globally.

CORE AND DEPLOYMENT PACKAGES

PACKAGE	WHEN	PURPOSE
Django 5.2.x	Day 1	Web framework, ORM, admin, users, tests
DRF 3.16.x	Day 1	Serializers, API views, auth, permissions, API tests
psycopg[binary]	Day 4	PostgreSQL driver used by Django
dj-database-url	Day 4	Reads a PostgreSQL DATABASE_URL safely
Gunicorn	Day 4	Production WSGI server used on Render
WhiteNoise	Day 4	Serves Django admin/static files in deployment

```
python -m pip install "Django>=5.2,<5.3" "djangorestframework>=3.16,<3.17"
python -m pip install "psycopg[binary]>=3.2,<4" gunicorn whitenoise dj-database-url
python -m pip freeze > requirements.txt
python -m pip check
```

CREATE THE GIT REPOSITORY

```
git init
git add .
git commit -m "Initialize Django project"
git branch -M main
```

REQUIRED .GITIGNORE ENTRIES

```
.venv/
__pycache__/
*.py[cod]
.env
db.sqlite3
staticfiles/
.DS_Store
```

BEFORE THE FIRST COMMIT

Confirm that `git status` does not list `.venv`, `.env`, database files, or secret values. If a secret is committed, rotating the secret is required; deleting the visible line is not enough.

CONFIGURE POSTMAN FOR THE TASK API

Postman supports manual endpoint exploration; it does not replace the automated Django tests written on Day 4.

CREATE THE COLLECTION AND ENVIRONMENT

- 1 Create a collection named **Task Management API**.
- 2 Create an environment named **Local Django**.
- 3 Add `base_url` with value `http://127.0.0.1:8000`.
- 4 Add `token` with an empty initial value.
- 5 Select the Local Django environment before sending requests.

REQUEST	METHOD + URL	BODY / AUTH	EXPECTED
Get token	POST {{base_url}}/api/auth/token/	JSON: {"username":"...", "password":"..."}	200 + token
List tasks	GET {{base_url}}/api/tasks/	Authorization: Token {{token}}	200 + array
Create task	POST {{base_url}}/api/tasks/	Token auth + JSON Task fields	201 + task
Update task	PATCH {{base_url}}/api/tasks/{id}/	Token auth + changed fields	200 + task
Delete task	DELETE {{base_url}}/api/tasks/{id}/	Token auth	204

SAVE THE TOKEN AUTOMATICALLY

In the token request, open the **Tests** or post-response script area and save the returned token into the active environment. The exact Postman interface label may change, so the instructor should demonstrate this once.

```
const data = pm.response.json();
pm.environment.set("token", data.token);
```

SECURITY RULE

Never export or share an environment that contains a real deployment token, password, database URL, or secret key. Use placeholders in the submitted collection.

POSTMAN VS. AUTOMATED TESTS

Postman is used to explore and demonstrate the API. DRF APITestCase is still required for repeatable authentication, CRUD, ownership, and validation tests in the repository.

DATABASE WORKFLOW - SQLITE TO POSTGRESQL

Start simple locally, then switch through an environment variable for deployment.

DAYS 1-4: SQLITE

- 1 Django creates `db.sqlite3` after `python manage.py migrate`.
- 2 Stop the Django server before opening the file in DB Browser when possible.
- 3 Use **Database Structure** to inspect tables and **Browse Data** to inspect rows.
- 4 Do not manually edit ownership or authentication tables during exercises; use Django admin, the API, or management commands.
- 5 Keep `db.sqlite3` out of Git. Each participant can generate a clean database from migrations.

DAY 4-5: NEON POSTGRESQL

- 1 Create a Neon project named `tuwaiq-django-api`.
- 2 Open the project's connection details and copy the PostgreSQL connection string.
- 3 Store it as an environment variable named `DATABASE_URL`; never paste it into `settings.py` or Git.
- 4 Install `psycopg[binary]` and `dj-database-url` in the project environment.
- 5 Update Django settings to keep SQLite as the local default and use PostgreSQL only when `DATABASE_URL` exists.

```
# settings.py
import os
import dj_database_url

# Keep Django's generated SQLite DATABASES setting above this block.
if os.getenv("DATABASE_URL"):
    DATABASES = {
        "default": dj_database_url.config(
            conn_max_age=600,
            conn_health_checks=True,
        )
    }
```

VERIFY POSTGRESQL SAFELY

```
# Run only in a terminal where DATABASE_URL is set.
python manage.py migrate
python manage.py check
python manage.py createsuperuser
```

WHY NO LOCAL POSTGRESQL INSTALLATION?

The managed Neon database is the production target. Avoiding a local PostgreSQL server removes password, port, service, and operating-system differences while still teaching the correct Django/PostgreSQL integration.

PREPARE THE PROJECT FOR FREE DEPLOYMENT

This page confirms the required tools and configuration; the final deployment activity remains part of Day 5.

PRODUCTION SETTINGS

```
# settings.py
SECRET_KEY = os.getenv("SECRET_KEY", "development-only-key")
DEBUG = os.getenv("DEBUG", "True").lower() == "true"
ALLOWED_HOSTS = [
    item.strip() for item in
    os.getenv("ALLOWED_HOSTS", "127.0.0.1,localhost").split(",")
    if item.strip()
]

STATIC_URL = "static/"
STATIC_ROOT = BASE_DIR / "staticfiles"
```

WHITENOISE

Place WhiteNoise immediately after Django's SecurityMiddleware so deployed admin/static assets are served correctly.

```
MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "whitenoise.middleware.WhiteNoiseMiddleware",
    # remaining middleware...
]
```

EXPECTED RENDER CONFIGURATION

RENDER SETTING	VALUE
Runtime	Python
Build command	pip install -r requirements.txt && python manage.py collectstatic --noinput && python manage.py migrate
Start command	gunicorn config.wsgi:application
DATABASE_URL	Neon connection string (secret)
SECRET_KEY	Long random value (secret)
DEBUG	False
ALLOWED_HOSTS	The assigned Render hostname

FREE SERVICE BEHAVIOR

Render free web services may sleep after 15 minutes without traffic. Open the service before a demonstration and allow time for the first request to wake it. The local filesystem is not persistent, which is why deployed data must use Neon PostgreSQL rather than SQLite.

FINAL READINESS CHECKLIST

A participant is ready only when every required check passes on the laptop they will bring.

CHECK	WINDOWS	MACOS	PASS RESULT
Python	py --version	python3 --version	Python 3.13.x
pip	py -m pip --version	python3 -m pip --version	Shows an installed path
VS Code	code --version	code --version	Prints a version
Git	git --version	git --version	Prints a version
Virtual env	.venv\Scripts\Activate.ps1	source .venv/bin/activate	Prompt begins (.venv)
Django	python -m django --version	python -m django --version	5.2.x
Server	python manage.py runserver	python manage.py runserver	Django page at 127.0.0.1:8000
Postman	Postman Echo GET	Postman Echo GET	200 OK
SQLite	Open DB Browser	Open DB Browser	App opens successfully
Accounts	GitHub + Neon + Render	GitHub + Neon + Render	Dashboards accessible

PARTICIPANT SUBMISSION BEFORE DAY 1

- Screenshot or text output showing Python, Git, and Django versions.
- Screenshot of installed VS Code extensions.
- Postman Echo response showing 200 OK.
- GitHub username and confirmation that the dashboard opens.
- Confirmation that Neon and Render accounts are accessible.
- Any unresolved error copied as text, not only as a screenshot.

INSTRUCTOR RELEASE GATE

Review setup evidence at least 48 hours before Day 1. Participants with unresolved setup problems should attend a short installation clinic rather than spending the first workshop session installing software.

COMMON SETUP PROBLEMS

Use the exact error message when asking for help. Avoid reinstalling everything before identifying the failing component.

PROBLEM	LIKELY CAUSE	ACTION
Python command not found	PATH/launcher missing or terminal opened before install	Restart terminal/computer; rerun installer with PATH/launcher enabled
Wrong Python in VS Code	System interpreter selected instead of .venv	Run Python: Select Interpreter and choose .venv
Activate.ps1 blocked	PowerShell execution policy	Set CurrentUser policy to RemoteSigned, then reopen terminal
pip installs globally	Virtual environment is inactive	Activate .venv; confirm prompt and python path before install
Port 8000 already used	Another server process is running	Stop the old process or runserver 8001
SQLite database locked	DB Browser and Django are writing concurrently	Close DB Browser or revert unsaved DB Browser edits
GitHub push rejected	Authentication or remote URL issue	Sign in through VS Code/browser; verify git remote -v
Postman returns 401	Token missing, expired, or incorrect prefix	Obtain a new token; send Authorization: Token value
Postman returns 404	Wrong URL, route, id, or ownership filtering	Check base_url, trailing slash, route, object id, and signed-in user
Render deploy fails	Build/start command, dependency, or variable missing	Read the first meaningful error in logs; verify requirements and variables
Neon connection fails	Incorrect/expired URL or environment value	Copy a fresh connection string; preserve sslmode; do not add spaces

GOOD SUPPORT REQUEST

Include: operating system and version, the exact command, the complete error text, what you expected, what changed immediately before the error, and the output of the relevant version command. Remove passwords and secrets first.

RECOVERY PLAN WHEN LOCAL SETUP FAILS

A failed local installation must not prevent a prepared participant from taking part in Day 1.

USE THE FIRST OPTION THAT WORKS

- 1 Copy the exact error text and use the troubleshooting table. Retry only the failed step, then repeat the environment check.
- 2 Attend the scheduled setup clinic at least 48 hours before Day 1. Bring the same laptop, charger, administrator credentials, and account access.
- 3 If the issue cannot be fixed safely, use a club-prepared backup laptop with the approved versions and a fresh participant account/profile.
- 4 If no backup laptop is available, the Technical Lead may assign an instructor-prepared browser-based development workspace. The workspace must be tested with this project before it is offered.
- 5 After recovery, repeat the practical readiness exercise and submit the same evidence as every other participant.

FAILURE	PRIMARY RECOVERY	OWNER	PROOF
Python/venv cannot run	Repair at setup clinic; otherwise use approved backup laptop	Technical Lead	Version + venv checks pass
VS Code installation blocked	IT-approved installation or approved backup laptop	Participant + support	Extensions and terminal work
Git/GitHub blocked	Browser sign-in support; temporary local commits until access is restored	Workshop support	Repository push succeeds
Postman cannot install	Use the approved Postman web client only if the instructor confirms the required agent/network access	Technical Lead	Echo request returns 200
Laptop unavailable	Assign a preconfigured club laptop before Day 1	Program coordinator	Participant signs readiness form

RECOVERY IS TEMPORARY

The participant still needs an individual GitHub repository and must understand every command used. A recovery environment removes installation risk; it does not remove the learning outcomes or final project requirements.

CLEAN-COMPUTER VALIDATION PROTOCOL

This real-world test must be completed before the guide is marked final or submitted as fully validated.

TESTER REQUIREMENTS

- The tester did not write or edit this guide.
- The test uses a clean or newly reset participant-style computer without the workshop tools already configured.
- At least one Windows test and one macOS test are strongly recommended because the commands differ.
- The tester follows only this guide; the author may observe but must not explain missing steps.
- Passwords, tokens, database URLs, and other secrets are excluded from screenshots and notes.

TEST SEQUENCE

- 1 Record the device, operating-system version, architecture, test date, and tester name.
- 2 Install every required application and extension from the official links.
- 3 Create or access the required free accounts.
- 4 Complete the readiness environment, knowledge check, practical exercise, and final checklist.
- 5 Record every unclear instruction, unexpected prompt, error, and workaround without help from the author.
- 6 Apply corrections to the guide, then repeat every failed step from a clean state.
- 7 The Technical Lead signs only when all required checks pass.

TEST	REQUIRED EVIDENCE	PASS RULE
Installations	Version outputs and installed-extension screenshot	All approved versions open/run
Django environment	Virtual environment, django version, check output	No identified issues
Local server	Browser showing 127.0.0.1:8000	Django success page loads
Postman	Echo request and response	HTTP 200
Accounts	Non-secret dashboard access confirmation	GitHub, Neon, Render accessible

NO INVENTED EVIDENCE

Document generation and visual PDF review are not substitutes for this test. Keep the status as **pending** until a real independent tester completes and signs the evidence record on the next page.

INDEPENDENT TEST EVIDENCE RECORD

Complete one copy per tested operating system and attach screenshots or a recording link.

FIELD	RECORDED VALUE
Tester name and role	_____
Independent of guide author?	Yes / No
Operating system + version	_____
Device architecture	Windows x64 / Apple silicon / Intel Mac / Other: _____
Clean-device method	New device / reset profile / virtual machine / other: _____
Test date and duration	_____
Evidence folder/link	_____

CHECKPOINT	PASS	ISSUE / CORRECTION
All required tools installed	Yes / No	_____
Accounts accessible	Yes / No	_____
Readiness project runs	Yes / No	_____
Postman returns 200	Yes / No	_____
Instructions completed without author help	Yes / No	_____
Guide corrected and failed steps retested	Yes / No / N/A	_____

FINAL SIGN-OFF

Overall result: PASS / FAIL Tester signature: _____ Date: _____
 Technical Lead approval: _____ Date: _____

TASK 3 SUBMISSION MAP

Use this page to prepare the five links requested by the Curriculum Lead.

SUBMISSION ITEM	CONTENT IN THIS GUIDE	LINK TO SUBMIT
Prerequisites	Entry prerequisites, required level, and what to bring	Guide link + page 2
Installation guide	Tools, accounts, Windows/macOS installation, dependencies, and project path	Guide link + installation pages
Environment check	Readiness environment, knowledge check, practical exercise, and final checklist	Guide link + readiness pages
Troubleshooting guide	Common problems plus recovery plan	Guide link + troubleshooting pages
Clean-computer test evidence	Completed independent evidence record and supporting screenshots	Separate evidence link after testing

FINAL COMPLETION GATE

- A new participant completes the guide without direct help.
- The readiness project runs and every environment check passes.
- Common problems can be solved using the documented actions.
- A working recovery environment is available before Day 1.
- Independent clean-computer evidence is completed and approved.
- The five submission links open for the Curriculum Lead and Technical Lead.

CURRENT VALIDATION STATUS

The written guide and evidence templates are ready. Final validation remains **pending** until the clean-computer and independent-tester records are completed with real results.

OFFICIAL DOWNLOADS AND REFERENCES

Use only these official project or vendor pages. Avoid third-party download mirrors.

RESOURCE	OFFICIAL URL
Python downloads	https://www.python.org/downloads/
Python virtual environments guide	https://packaging.python.org/en/latest/guides/installing-using-pip-and-virtual-environments/
Django 5.2 release notes and Python support	https://docs.djangoproject.com/en/5.2/releases/5.2/
Django REST Framework 3.16 announcement	https://www.django-rest-framework.org/community/3.16-announcement/
Visual Studio Code downloads	https://code.visualstudio.com/download
Microsoft Python extension	https://marketplace.visualstudio.com/items?itemName=ms-python.python
Git downloads	https://git-scm.com/install/
Postman installation	https://learning.postman.com/docs/getting-started/installation/install-app/
Postman Free plan	https://www.postman.com/pricing/
DB Browser for SQLite	https://sqlitebrowser.org/dl/
GitHub Free repositories	https://github.com/pricing
PostgreSQL downloads and license	https://www.postgresql.org/download/
Neon	https://neon.com/
Render free services	https://render.com/docs/free

VERSION POLICY

Use the latest security/bug-fix release within the approved Python 3.13, Django 5.2, and DRF 3.16 series. The instructor should validate the full stack once, freeze the tested requirements file, and distribute that same file to every participant.

FREE-PLAN POLICY

This guide was revised on September 30, 2026. Hosted service limits can change. Re-check the official Render, Neon, GitHub, and Postman plan pages before participant onboarding and record the clean-computer validation separately.